

Documentazione dell'Interprete di Permutazioni

Introduzione al DSL del progetto

Il DSL scelto per questo progetto è pensato per consentire la manipolazione e l'analisi di permutazioni finite, espresse in notazione ciclica. L'interfaccia consente la definizione, la trasformazione e la visualizzazione delle permutazioni, con un focus didattico e matematicamente rigoroso (Sarebbe, per esempio, un utile strumento per accompagnare lo studio di uno studente alle prime prese con la struttura dei gruppi simmetrici).

Scrittura in cicli: formato di input delle permutazioni

La notazione convenzionale adoperata dall'interprete per indicare le permutazioni finite è quella della scrittura in cicli algebrica. Tale notazione nasce dall'idea che una permutazione degli elementi di un insieme finito può essere descritta da un'espressione di soli numeri a parentesi senza dover ricorrere a diagrammi o grafi. Fissata una permutazione su un insieme finito, per ricavarne la scrittura in cicli algebrica, è sufficiente numerare arbitrariamente gli oggetti dell'insieme in maniera da identificarli con i numeri da 1 a n (dove quest'ultimo è la cardinalità finita dell'insieme). A questo punto per ricavare la scrittura in cicli si procede in maniera algoritmica (*nel seguente elenco 'a' e 'b' sono numeri*):

1. si scrive "(".
2. si sceglie un numero a (non scelto in precedenza) e lo si scrive dopo "(".
3. si guarda dove la permutazione manda l'oggetto ' a ', sia questo ' b ', allora si scriverà ' b ' subito a destra di ' a '.
4. si ripete il punto precedente finché non si riottiene ' a ' (il primo numero scelto) a questo punto, ' a ' non va riscritto e si scrive invece ")".
5. si ripetono i punti precedenti finché tutti i numeri da 1 a n

Per costruzione si ottiene un'espressione costituita da successioni di numeri incapsulate da parentesi dove queste ultime prendono il nome di cicli, da cui il nome della notazione. La comodità della scrittura in cicli algebrica è che per ogni numero è immediato sapere dove

questo viene mandato dalla permutazione: è sufficiente guardare il numero a destra nel relativo ciclo (se si è scelto il numero finale di un ciclo si guarderà il primo dello stesso ciclo).

Esempio: la permutazione che manda 1 a 2, 2 a 3 e 3 a 1 (e lascia invariato 4) si scrive:

$$(1\ 2\ 3)(4)$$

Ogni ciclo rappresenta un percorso chiuso: l'elemento iniziale viene mappato sul secondo, il secondo sul terzo e quest'ultimo torna al primo mentre il quarto è fermo.

Nota: Nell'interprete, anche i **punti fissi** (cioè elementi che restano invariati) vengono esplicitamente mostrati nella notazione ciclica come cicli formati da singoli elementi, come lo era (4) nell'esempio precedente. È importante notare che il significato della permutazione si dedurrebbe ugualmente anche se i cicli di singoli elementi fossero omessi, tuttavia, con tale scelta, non si capirebbe più la cardinalità dell'insieme su cui la permutazione agisce (da questo la mia scelta di inserire i cicli di un elemento).

Dichiarazioni, aggiornamenti e prodotti

È possibile definire nuove permutazioni assegnando un identificatore a una scrittura in cicli algebrica o un comando/espressione che restituisca una permutazione e successivamente si potranno eseguire varie operazioni su questa o definire nuove permutazioni a partire dalla prima. Per farlo si adopera la seguente sintassi per le dichiarazioni:

```
1 var p = (1 2 3)(4)
2 var q = power(p, 2)
3 var r = conjugate(p, q)*Inverse(p)*Inverse(q)
```

Oltre la dichiarazione di variabili permutazione è anche possibile definire variabili intere, booleane oppure funzioni espresse come lambda astrazioni utilizzando l'analoga sintassi con l'operatore var. In particolare la dichiarazione di variabili booleane e intere può essere fatta assegnando intere espressioni agli identificatori (contenenti gli ovvi operatori tra interi/-booleani o anche oggetti che restituiscono interi o booleani). Si può dare il seguente esempio chiarificatore:

```
1 var p = 3          var p = 3 + 4 * 2 + order(permutazione)
2 var q = true       var q = true or false and parity(permutazione)
3 var r = lambda(a,b) = if a then power(b,2) else power(b,3)
```

l'aggiornamento di variabili, laddove possibile si effettua tramite l'operatore "←", come mostrato dai seguenti esempi:

```
1 var numero = 1; numero <- numero + 1;    var p = (1 2); p <- (1 2 3)
   ;
```

l'unico altro comando che come la dichiarazione e l'aggiornamento non ha un nome esplicito - e dunque non compare né nella successiva sezione, né nell'help menu dell'interprete - è il prodotto di composizione. Esso è implementato in maniera equivalente ad un prodotto di interi e dunque è invocato dal simbolo * e si aspetta di essere preceduto e seguito da oggetti permutazione ovunque lo si applichi, come ad esempio in una semplice dichiarazione:

`varpermutazione1 = permutazione2 * permutazione3`. Ha l'effetto di assegnare `permutazione1` come la composizione di `permutazione2` e `permutazione3` nell'ordine in cui sono scritte (la composizione non è commutativa). **Una particolarità aggiuntiva** dell'implementazione del prodotto è che esso può essere calcolato anche attraverso un comando apposito che è `productVisualize`. Questo comando, oltre che eseguire il prodotto, ha la particolarità ulteriore di creare una finestra tramite apposite librerie python in cui è visualizzabile un'animazione rappresentativa della composizione di permutazioni. Invocando il comando con la sua sintassi:

```
varpermutazione1 = productVisualize(perm1, perm2, ..., permk)
```

verrà calcolato il prodotto delle permutazioni date in argomento e verrà visualizzata la composizione delle permutazioni tramite diagrammi a treccia.

Nota: come deducibile dai precedenti esempi, la scrittura di istruzioni successive deve essere opportunamente separata dal simbolo ";"

Struttura dei comandi

Ogni comando dell'interprete è pensato per essere immediato da capire, tuttavia, per maggiore chiarezza è stato inserito un comando `help` con 2 sintassi alternative:

SINTASSI1: `help` SINTASSI2: `help comando`

La prima delle due sintassi scrive l'elenco di tutti i comandi disponibili nel DSL esclusi le dichiarazioni, gli aggiornamenti e i prodotti. La seconda scrive invece la sintassi e il funzionamento del comando dato in input.

Di seguito è fornita una panoramica, estratta dall'help menu dell'interprete completa di ogni comando disponibile con relativa sintassi e funzionamento, il tutto elencato alfabeticamente.

Comandi disponibili: help menu

clear `clear`

Elimina tutte le variabili utente e ripristina l'ambiente allo stato iniziale.

list `list`

Mostra le variabili attualmente definite e i loro valori.

print `print(permutazione)`

Stampa la permutazione sia come lista di interi sia in forma algebrica.

let `let nome = expr1 in expr2`

Valuta `expr2` nell'ambiente esteso con `nome` legato al valore temporaneo `expr1`.

- if** `if cond then ... else ... endif`
Valuta `cond` e, se vero, esegue il blocco `then`, altrimenti il blocco `else`.
- while** `while cond do ... done`
Finché `cond` è vero, ripete il corpo del loop.
- lambda**
 `lambda(param1,...,paramN) expr`
Crea una funzione anonima con i parametri dati.
- braidDiagram**
 `braidDiagram(nomeVariabile)`
Mostra il diagramma a treccia della permutazione.
- cyclesDiagram**
 `cyclesDiagram(nomeVariabile)`
Mostra il diagramma a cicli della permutazione in input.
- cyclesNumber**
 `cyclesNumber(permutazione)`
Stampa il numero di cicli della permutazione in input *e restituisce il rispettivo intero*.
- fix** `fix(permutazione)`
Stampa i punti fissi della permutazione in input *e restituisce il rispettivo intero*.
- identity**
 `identity(n)`
Genera la permutazione identità su `n` elementi *e la restituisce*.
- inverse**
 `inverse(permutazione)`
Calcola l'inversa della permutazione in input *e la restituisce*.
- power** `power(permutazione, esponente)`
Eleva la permutazione in input all'esponente intero non negativo in input *e restituisce la permutazione risultante*
- conjugate**
 `conjugate(q, s)`
Calcola $q \cdot s \cdot q^{-1}$ *e restituisce la permutazione risultante*.
- order** `order(permutazione)`
Stampa l'ordine della permutazione (calcolato come mcm delle lunghezze dei cicli) *e restituisce l'intero risultante*
- parity** `parity(permutazione)`
Stampa se la permutazione è pari o dispari *e restituisce il booleano vero se la permutazione è pari e il booleano falso altrimenti*.

shift `shift(permutazione, offset)`

Trasla tutti gli elementi della permutazione in input di un offset positivo o negativo e restituisce la permutazione risultante

productVisualize

`productVisualize(perm1, perm2, ..., permk)`

Restituisce la permutazione prodotto $\text{perm}_1 * \text{perm}_2 * \dots * \text{perm}_k$ e mostra un'animazione per ogni passo di composizione.

Scelte di progettazione

Parsing, trasformazione AST e Lark

L'analisi sintattica è affidata a Lark, che genera un parse tree a partire dalla grammatica definita in `grammar`. Il parser è istanziato con:

```
parser = Lark(grammar, start="program")
```

Gli errori di parsing (token inattesi, caratteri non riconosciuti, altri errori sintattici) sono segnalati tramite le eccezioni standard di Lark: `UnexpectedToken`, `UnexpectedCharacters`, `LarkError`. Non si utilizza più una `ParsingError` personalizzata.

Dalla struttura intermedia (parse tree) si ottiene l'AST mediante tre semplici funzioni:

- `transform_expression(tree)`: converte i nodi di espressione (`Apply`, `Lambda`, `Var`, `CycleList`) in istanze delle rispettive classi AST;
- `transform_command_tree(tree)`: trasforma ciascun comando (`Assign`, `VarDecl`, `Print`, `IfElse`, `While`, diagrammi, ecc.) in un nodo AST dedicato;
- `transform_command_seq_tree(tree)`: costruisce ricorsivamente sequenze di comandi tramite il nodo `CommandSequence(first, rest)`.

Ambiente, store e state

L'architettura della memoria è ripartita in due moduli:

Environment mappa (immutabile) da identificatori a `DVal` (locazioni o closure). Si crea un ambiente vuoto con `empty_environment()` e lo si estende con `bind(env, name, value)`;

State dataclass che contiene:

- **store**: funzione immutabile da indirizzi interi (`Loc`) a valori (`MVal`);
- **next_loc**: contatore della prossima locazione libera.

Si inizializza con `empty_state()`, si aggiorna con `allocate(state, value)` e `update(state, loc, value)`.

Scoping lessicale block-local

Ogni blocco (`IfElse`, `While`, `Let`) crea un ambiente temporaneo:

- In `IfElse` e `While`, `next_loc` viene salvato all’inizio del blocco e ripristinato alla fine, mentre lo `store` mantiene solo le locazioni ancora rilevanti;
- Il costrutto-espressione `let` consente di legare temporaneamente una variabile a un valore tramite la sintassi `let x = expr1 in expr2`. La variabile `x` è visibile solo all’interno di `expr2`, secondo uno scoping statico e block-local. Al termine della valutazione, il binding viene eliminato, evitando effetti collaterali e occupazione permanente di memoria. Questo costrutto è utile per definire sotto-calcoli locali senza dichiarazioni globali.

Parser ed evaluator: funzioni principali

Parser

- `Lark(grammar, ...)`: costruisce il parser;
- `transform_expression`, `transform_command_tree`, `transform_command_seq_tree`: generano l’AST.

Evaluator L’evaluator é organizzato in tre funzioni:

- `evaluate_expr(node, env, state) -> Eval`: usa `match-case` per i nodi di espressione (`CycleList`, `Number`, `Bool`, `FunctionApp`, `Apply`);
- `execute_command(cmd, env, state) -> (Environment, State)`: usa `match-case` per i comandi (`VarDecl`, `Assign`, `Print`, `IfElse`, `While`, diagrammi, ecc.);
- `execute_command_seq(seq, env, state)`: chiama `execute_command` su ogni comando della sequenza.

Operatori built-in come valori di prima classe

Gli operatori primitivi (`+`, `-`, `*`, `/`, `=`, `<`, `>`, `≥`, `≤`, ecc.) sono istanze di `Operator(type, fn)` legate all’ambiente iniziale tramite `bind`. Sono trattati come normali valori:

- passabili come argomenti e restituiti come risultati;
- facilmente estendibili senza modifiche all’evaluator;
- controllati nell’arietà e nei tipi a runtime.

Gestione degli errori

- Errori di parsing: eccezioni di Lark con messaggi dettagliati su posizione e tipo di token;
- Errori semantici: sollevati come `ValueError` o `TypeError` in `evaluate_expr` e `execute_command` (variabili non definite, tipo errato, numero di argomenti scorretto, tentativo di applicare un non-funzione, ecc.).

Funzionalità grafiche

Le funzioni `braidDiagram` e `cyclesDiagram` forniscono visualizzazioni dinamiche e statiche delle permutazioni, basate su `matplotlib` e `networkx`. Tra le altre funzionalità grafiche c'è anche la funzione `anima_treccia_morph` implementa un'animazione continua tra due configurazioni di diagrammi a treccia ed è alla base del già citato comando `productVisualize`.

Programmi di testing

Programma di test 1: Esplorazioni iniziali da parte di un neofita dell'algebra

Il seguente script simula un'interazione con l'interprete da parte di uno studente alle prime armi con l'algebra delle permutazioni. L'utente fa largo uso del comando `help` per esplorare i comandi disponibili, definisce permutazioni, ne analizza l'ordine, la parità, i punti fissi, le visualizza e ne studia alcune operazioni strutturali.

```
1 # L utente comincia chiedendo aiuto per capire cosa puo' fare
2 help
3
4 # incuriosito dal comando braidDiagram, ne legge la descrizione
5 help braidDiagram
6
7 # Allo stesso modo vuole capire come usare il comando per generare
  l identit
8 help identity
9
10 # Definisce una permutazione identit su 5 elementi per partire
    da qualcosa di noto
11 var id5 = identity(5)
12
13 # La stampa per verificarne la forma
14 print(id5)
15
16 # Prova ora a definire una permutazione semplice manualmente
17 var sigma = (1 2 3)
```

```
18
19 # La stampa per capirne struttura e lista
20 print(sigma)
21
22 # L utente decide di visualizzarla tramite un diagramma a treccia
23 braidDiagram(sigma)
24
25 # E anche con un diagramma a cicli
26 cyclesDiagram(sigma)
27
28 # Vuole sapere se questa permutazione ha punti fissi
29 fix(sigma)
30
31 # Indaga se      pari o dispari
32 parity(sigma)
33
34 # Si chiede quanti cicli siano presenti (dovrebbe essere uno solo)
35 cyclesNumber(sigma)
36
37 # Si chiede quale sia l ordine di questa permutazione
38 order(sigma)
39
40 # Gioca un p o con la potenza della permutazione
41 sigma2 = power(sigma,2)
42 print(sigma2)
43
44 # Visualizza il prodotto in treccia
45 braidDiagram(sigma2)
46
47 # Prova a fare la potenza 3 e nota che deve tornare a l l identit
48 var sigma3 = power(sigma,3)
49 print(sigma3)
50
51 # Gioca con il coniugato: costruisce una permutazione tau
52 var tau = (1 3)
53 print(tau)
54
55 # Ora costruisce il coniugato di sigma tramite tau
56 var conj = conjugate(tau,sigma)
57 print conj
58
59 # Anche il diagramma del coniugato lo incuriosisce
60 braidDiagram(conj)
61
62 # Lista finale delle permutazioni definite
63 list
64
65 # Infine, resetta l ambiente
```



```
66 clear
```

Programma di test 2: conti di un più esperto conoscitore della struttura dei gruppi simmetrici

questo programma simula l'utilizzo dell'interprete da parte di uno studente con conoscenze di algebra solide che cerca di verificare risultati teorici sul sottogruppo di Klein del gruppo simmetrico S_4 .

```

1 # Definiamo i generatori classici del sottogruppo di Klein in S_4
2 var a = (1 2)(3 4)
3 var b = (1 3)(2 4)
4
5 # Verifichiamo che entrambi siano involuzioni
6 power(a, 2)          # deve dare identit
7 power(b, 2)          # idem
8
9 # Calcoliamo il prodotto ab
10 var ab = a * b
11 print(ab)           # dovrebbe essere (1 4)(2 3)
12
13 # Verifichiamo che ab sia un'involuzione anche lui
14 power(ab, 2)
15
16 # Osserviamo l'ordine dei generatori e del prodotto
17 order(a)
18 order(b)
19 order(ab)           # tutti devono avere ordine 2      gruppo abeliano
20
21 # Verifica se le permutazioni sono pari
22 parity(a)
23 parity(b)
24 parity(ab)          # tutte devono essere pari        sottogruppo di A_4
25
26 # Coniugazione: a^{-1} * b * a = b? (se s      abeliano)
27 var conj = conjugate(a, b)
28 print(conj)         # deve dare b se il gruppo      abeliano
29
30 # Visualizziamo tutte le permutazioni
31 braidDiagram(a)
32 braidDiagram(b)
33 braidDiagram(ab)
34 cyclesDiagram(ab)
35
36 # Ora costruiamo H = <a, b>
37 # Mostriamo che {id, a, b, ab} ha chiusura per l'operazione di
    composizione

```

```
38 var id = identity 4
39 var aba = a * ab
40 var bab = b * ab
41
42 print(aba)           # dovrebbe dare b
43 print(bab)           # dovrebbe dare a
44
45 # Verifica che tutti i composti restituiscano elementi del gruppo
46 order(aba)           # ordine di b
47 order(bab)           # ordine di a
48
49 # Shift per costruire un sottogruppo isomorfo al klein in S_6
50 a <- shift(a, 2)
51 b <- shift(b, 2)
52 var ab_shifted = a * b
53 print(ab_shifted)
54
55 # Lista finale
56 list
```

Programma di test 3: giochi di visualizzazione

Questo programma simula l'utilizzo dell'interprete da parte di un utente che è interessato unicamente a creare diagrammi colorati e intriganti visivamente.

```
1 # Definizione di permutazioni fortemente intrecciate e piene di
  cicli
2 var sigma = (1 8 3 6)(2 7 4)(5 9)
3 var tau = (1 5 2 6)(3 8)(4 7)(9 10)
4 var rho = (2 4 6 8)(1 3 5 7)(9 10)
5
6 # Prima visualizzazione: treccia e cicli per ciascuna
7 braidDiagram(sigma)
8 cyclesDiagram(sigma)
9
10 braidDiagram(tau)
11 cyclesDiagram(tau)
12
13 braidDiagram(rho)
14 cyclesDiagram(rho)
15
16 # Visualizzazione morphing tra prodotti
17 var alpha = productVisualize(sigma, tau)
18 var beta = productVisualize(tau, rho)
19 var gamma = productVisualize(sigma, rho)
20
21 # Altra permutazione interessante
```

```
22 theta = (1 4 7 2 5 8)(3 6 9)(10 12 11)
23 braidDiagram(theta)
24 cyclesDiagram(theta)
25
26 # Prova con traslazione negativa e positiva
27 theta <- shift(theta, 3)
28 braidDiagram(theta)
29
30 rho <- shift(rho, -1)
31 cyclesDiagram(rho)
32
33 # Coniugazione di permutazioni a supporto ampio: cambia struttura
    mantenendo tipo
34 var omega = conjugate(tau, sigma)
35 braidDiagram(omega)
36 cyclesDiagram(omega)
37
38 # Potenze di permutazioni: auto-interazioni cicliche
39 var tau2 = power(tau, 2)
40 var gamma3 = power(gamma, 3)
41 braidDiagram(gamma3)
42
43 # ancora composizioni ma post-shift
44 var delta = productVisualize(rho,theta,rho,omega,rho)
```

Programmi ulteriori

In questa sezione vengono proposti ulteriori 3 programmi a tema incentrati su dichiarazione di funzioni, costruito if e costruito while rispettivamente. Questi sono presentati senza una narrazione spiegata da commenti come i precedenti programmi e sono invece scritti in una forma più snella come semplici liste di istruzioni. Il loro scopo è appunto quello di mostrare come l'interprete gestisce dichiarazione di funzioni, costruito if e costruito while i quali costituivano funzionalità non ancora presentate prima.

Programma 1

```
1 var ide = (1 4 5)(3 2 8)(9 6 7);
2 print(ide);
3 var inv = lambda(p) inverse(p);
4 var twice = lambda(f,p) f(f(p));
5 print(inv(ide));
6 print(twice(inv,ide));
7 var p2 = power(ide,2);
8 print(p2);
9 var p3 = power(ide,3);
```

```
10 print(p3);
11 print(let inc = lambda(n) shift(identity(n),1) in inc(5));
12 var p = ide;\#
13 var q = p2;
14 var r = p3;
15 print(let comp = lambda(p,q,r) conjugate(p,conjugate(q,r)) in comp(p
    ,q,r));
16 var compose = lambda(f,g,p) f(g(p));
17 print(compose(inv,inv,ide));
```

Programma 2

```
1 var a = (1 2 3)(4 5 6)(7 8 9);
2 var b = shift(a,2);
3 var par = parity(b);
4 var sel = a;
5 if par then sel <- b else sel <- a endif;
6 print(sel);
7 var c = conjugate(b,shift(a,1));
8 var ord_c = order(c);
9 var comp = identity(6);
10 if ord_c > 3 then comp <- power(c,2) else comp <- identity(6) endif;
11 print(comp);
12 print(let d = power(b,5) in d);
13 print(let e = inverse(power(b,5)) in e);
14 var f = fix(c);
15 var numCycles = cyclesNumber(c);
16 var sel2 = b;
17 if numCycles == 2 then sel2 <- identity(6) else sel2 <- b endif;
18 print(sel2);
19 list;
```

Programma 3

```
1 var n = 5;
2 var fact = 1;
3 var i = 1;
4 while i <= n do fact <- fact * i; i <- i + 1; done;
5 print(fact);
6 var x = 10;
7 var sum = 0;
8 while x > 0 do sum <- sum + x; x <- x - 1; done;
9 print(sum);
10 var m = 3;
11 var pwr = 1;
```

```
12 while m > 0 do pwr <- pwr * 2; m <- m - 1; done;  
13 print(pwr);
```